

CS 400/600 – Data Structures

Abstract Data Types and a review of
C++ programming concepts

Abstract Data Types

Abstract Data Type (ADT): a definition for a data type solely in terms of a set of values and a set of operations on that data type.

Each ADT operation is defined by its inputs and outputs.

Encapsulation: Hide implementation details.

Data Structure

- A data structure is the physical implementation of an ADT.
 - Each operation associated with the ADT is implemented by one or more subroutines in the implementation.
- Data structure usually refers to an organization for data in main memory.

SimpleList

- Values: integers (to start with)
- Operations:
 - `bool Slist.insertfront(int i)`
 - `bool Slist.insertend(int i)`
 - `bool Slist.getfirst(int &i)`
 - `bool Slist.getlast(int &i)`
 - `void Slist.clear()`

SimpleList Implementation

- Implement as an unsorted single-linked list

```
class Lnode {  
public:  
    int value;  
    Lnode *next;  
    explicit Lnode(int newvalue = 0) {value = newvalue;  
                                     next = nullptr;}  
};
```

SimpleList Implementation

```
class SList {  
private:  
    Lnode* head;  
public:  
    SList() {head = nullptr;}  
    ~SList() {clear();}  
    bool insertfront(int i);  
    bool insertend(int i);  
    bool getfirst(int &val);  
    bool getlast(int &val);  
    void clear();  
    void printList() const;  
  
};
```

SimpleList Implementation

```
bool SList::insertfront(int i) {
    Lnode* newnode = new Lnode(i);
    newnode->next = head;
    head = newnode;
    return true;
}

bool SList::insertend(int i) {
    Lnode* newnode = new Lnode(i);
    Lnode* last = head;
    if (head == NULL) {
        head = newnode;
        return true;
    }
    while (last->next != NULL) {
        last = last->next;
    }
    last->next = newnode;
    return true;
}
```

SimpleList Implementation

```
bool SList::getfirst(int &val) {
    Lnode* oldhead = head;
    if (head == NULL) {return false;}
    val = head->value;
    head = head->next;
    delete(oldhead);
    return true;
}

void SList::clear() {
    Lnode* oldnode;
    while (head != NULL) {
        oldnode = head;
        head = head->next;
        delete(oldnode);
    }
}
```


SimpleList Implementation

```
bool SList::getlast(int &val) {
    if (head == NULL) {
        return false; // list is empty
    }

    Lnode* oldhead = head;

    // If there's only one node
    if (head->next == NULL) {
        val = head->value;
        delete head;
        head = NULL;
        return true;
    }

    // Traverse to the second last node
    while (oldhead->next->next != NULL) {
        oldhead = oldhead->next;
    }

    val = oldhead->next->value;
    delete oldhead->next;
    oldhead->next = NULL;

    return true;
}
```

SimpleList Implementation

```
void SList::printList() const {
    const LNode* current = head;
    bool first = true;

    while (current != NULL) {
        if (!first) {
            cout << ", ";
        }
        cout << current->value;
        first = false;
        current = current->next;
    }

    cout << endl;
}
```

Using the SimpleList

```
SList mylist;  
int   val;  
  
mylist.insertfront(7);  
mylist.insertfront(3);  
mylist.insertend(12);  
  
cout << "Here's the list: ";  
  
while (mylist.getfirst(val)) {  
    cout << val << ", ";  
}  
  
cout << endl;
```

Extending the class

- What if we want to be able to store a list of any type of data type/class?
- We can make this into a *template* to allow the class to be filled in later.

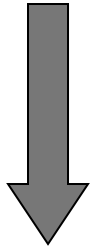
```
class LNode {  
public:  
    int value;  
    LNode *next;  
    LNode(int newvalue = 0) {value = newvalue; next = NULL;}  
}
```



```
template <class Elem> class LNode {  
public:  
    Elem data;  
    LNode *next;  
    LNode(const Elem& newvalue) {data = newvalue; next = NULL;}  
}
```

Using the template

```
LNode<double> node(3.14);
```



```
Class LNode<double> {  
public:  
    double data;  
    LNode<double> *next;  
    LNode<double>(double& newvalue) {data = newvalue;  
                                     next = NULL;}  
}
```

Defining an ADT

- C++'s *abstract classes* are a good way to define an ADT:

```
// List abstract class
template <class Elem> class List {
public:
    // Reinitialize the list. The client is responsible for
    // reclaiming the storage used by the list elements.
    virtual void clear() = 0;
    // Insert an element at the front of the right partition.
    // Return true if successful, false if the list is full.
    virtual bool insert(const Elem&) = 0;
    // Append an element at the end of the right partition.
    // Return true if successful, false if the list is full.
    virtual bool append(const Elem&) = 0;
    // Remove the first element of right partition. Return
    // true if successful, false if right partition is empty.
    // The element removed is returned in the parameter.
    virtual bool remove(Elem&) = 0;
    // Place fence at list start, making left partition empty
    virtual void setStart() = 0;
    ...
};
```

Defining an ADT

```
// List abstract class, continued...
// Place fence at list end, making right partition empty
virtual void setEnd() = 0;
// Move fence one step left; no change if already at start
virtual void prev() = 0;
// Move fence one step right; no change if already at end
virtual void next() = 0;
// Return length of left partition
virtual int leftLength() const = 0;
// Return length of right partition
virtual int rightLength() const = 0;
// If pos or more elements are in the list, set the size
// of left partition to pos and return true. Otherwise,
// do nothing and return false.
virtual bool setPos(int pos) = 0;
// Return in first parameter the first element of the
// right partition. Return true if successful, false
// if the right partition is empty.
virtual bool getValue(Elem&) const = 0;
// Print the contents of the list
virtual void print() const = 0;
};
```